

1

a

This reduction is quite similar to the weighted interval scheduling to shortest paths.

We start with creating n nodes and we are trying to find the shortest path from node 1 to node n . We can attach arcs from node i to node $i + 1$ with cost 0 to ensure a feasible solution. Then for each arc (i, j) in the arc set A , we put an arc from node i to node $j + 1$ (with the exception of $j = n$, in which case the arc goes to the node n) with a cost of -1 . Thus when trying to find the shortest path, the shortest path algorithm will take as many of these arcs with cost -1 which is the same as trying to find the most amount of arcs in a set that do not conflict.

We know arcs are non conflicting in the shortest paths algorithm because in order for an arc (i_1, j_1) to be conflicting with another arc (i_2, j_2) then $i_1 \leq i_2 \leq j_1 \leq j_2$. In the case of our graph construction, it is not possible to take arcs that satisfy these inequalities because the only way for a path to take 2 arcs that conflict is if $j_1 = i_2$, which is why the arcs go from i_1 to $j_1 + 1$ to prevent a path from taking conflicting arc. In other words, we need the $j + 1$ because unlike the WIS problem we are not allowed to have an arc that stops and then use another arc on the same node. Thus the $+1$ for the ending part of an arc forces the paths to only take arcs once it is no longer conflicting with the previous arc taken.

Additionally given the extra constraint of $\forall$ arcs, $i \leq j$, then we know that taking any path allows increases our node number essentially moving “forward” so we know that we never have a cycle allowing for a negative infinite path. To finish the reduction, the optimal objective would be equal to the most amount of non conflicting arcs in a set (times -1). The actual arcs chosen would be the arcs $i, j + 1$ that are part of the shortest path.

b

2

a

We can better visualize this problem as Ay where the individual rows of this matrix add up to u and the individual columns add up to v . Specifically the sum of the first row of Ax should be equal to u_1 and so on (same with columns). Now our goal is to prove that if Bx has a solution (given the same row and column summation constraints) then Ay must also have a solution.

This is quite simple since $\forall a_{ij} \neq 0, b_{ij} = 1 = a_{ij}/a_{ij}$. Thus, $b_{ij}x_{ij} = a_{ij}y_{ij} \rightarrow x_{ij} = a_{ij}y_{ij}$.

NOT FINISHED

b

We can construct the graph as a bipartite one where V_1 consist of p nodes and V_2 consists of q nodes. The nodes in V_1 correspond to the rows of Bx and the nodes in V_2 correspond to the columns of Bx . We assign arcs from each node in V_1 to each node in V_2 . These arcs then represent x_{ij} where i is the row and j is the column. By our feasibility shown in **2.a**, we know that these flows x_{ij} can correspond to a feasible solution y_{ij} . These arcs have o lower or upper bound, since this is a circulation feasibility problem we can assume all 0 costs.

To induce a flow, we can create a new source node which has outgoing arcs to all the nodes in V_1 and all the nodes in V_2 point to the source node. Each of the outgoing arcs from the source to a node $i \in V_1$ has a lower bound of u_i , and similarly an arc from node $j \in V_2$ to the source node has a lower bound of v_j . Since we are given that $\sum_{i=1}^p u_i = \sum_{j=1}^q v_j$ we know that these ingoing and outgoing arcs will lead to a 0 circulation.

Since the circulation problem ends with all nodes having a 0 supply excess, we know that since each node $i \in V_1$ have an excess of u_i so $u_i - \sum_{j=1}^q x_{ij} = 0$. Similarly, since each node $j \in V_2$ have a deficit of v_j so $-v_j + \sum_{i=1}^p x_{ij} = 0$. Thus if there is a feasible solution then the values of x_{ij} directly correspond to a feasible solution of x_{ij} in **2.a** which was discussed to prove a feasible solution in the original problem.

3

The scheduling reduction fails quite similarly to the one discussed in class, except we also have to partition the intervals on the differing machines active. Thus we can compute the total compute power of an interval as the length of the interval multiplied by the number of machines in that interval (constant since we partitioned on machine intervals).

i	1	2	3	4
p_i	1.5	1.25	2.1	3.6
r_i	3	1	3	5
r_i	5	4	7	9

With machines as

Interval	Num Machines
[1,3]	3
[3,4]	2
[4,6]	3
[6,7]	2

Interval	Num Machines
[7,11]	4

We build off of these intervals to now partition on the different intervals of jobs active during those times and the total processing time at each interval

Interval	Num Machines	Jobs Active	Interval Length	Max Processing Power
[1,3]	3	2	3	9
[3,4]	2	1,2,3	1	2
[4,5]	3	1,3	1	3
[5,6]	3	3,4	1	3
[6,7]	2	3,4	1	2
[7,9]	4	4	2	8